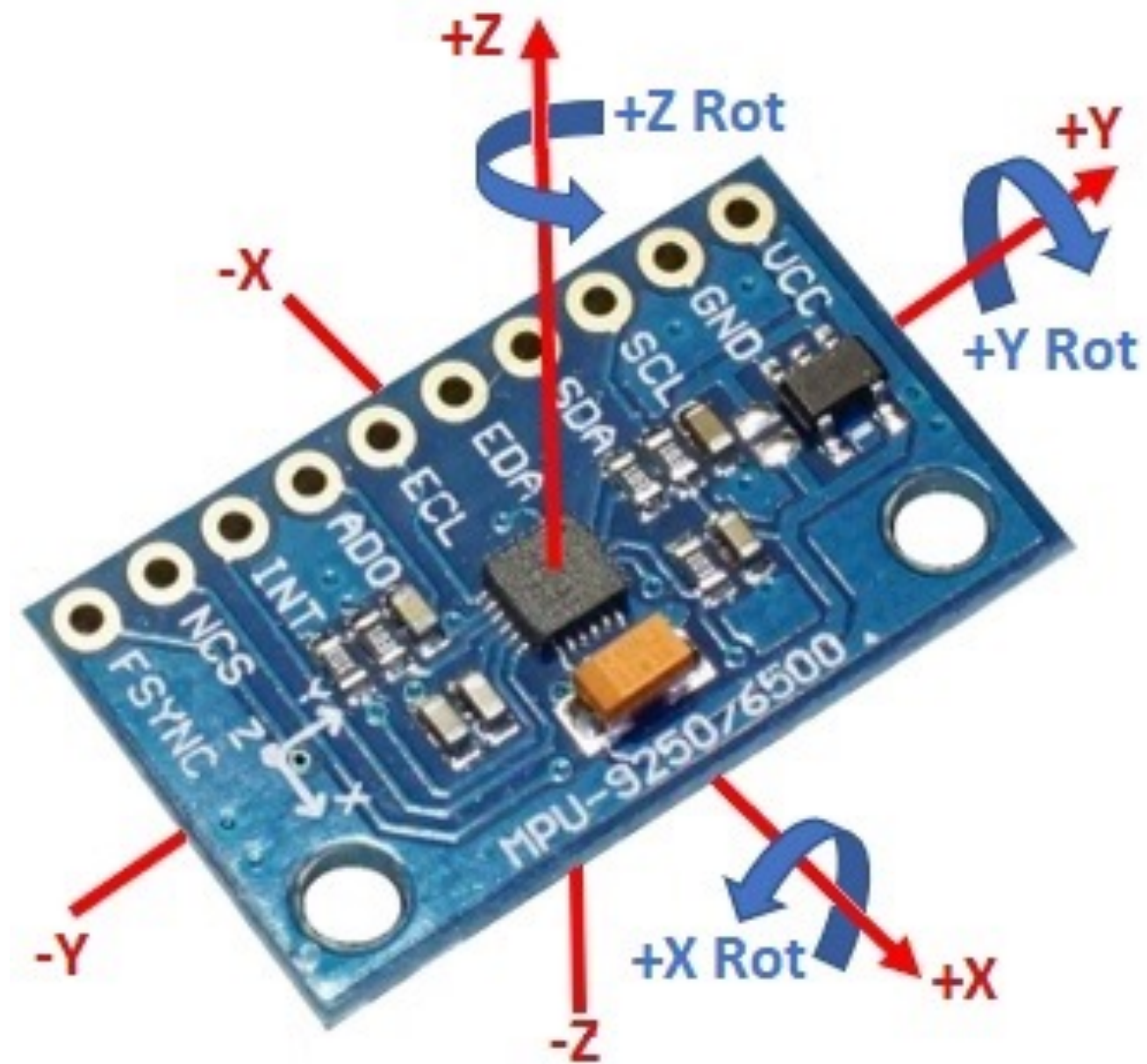


Second Milestone Presentation

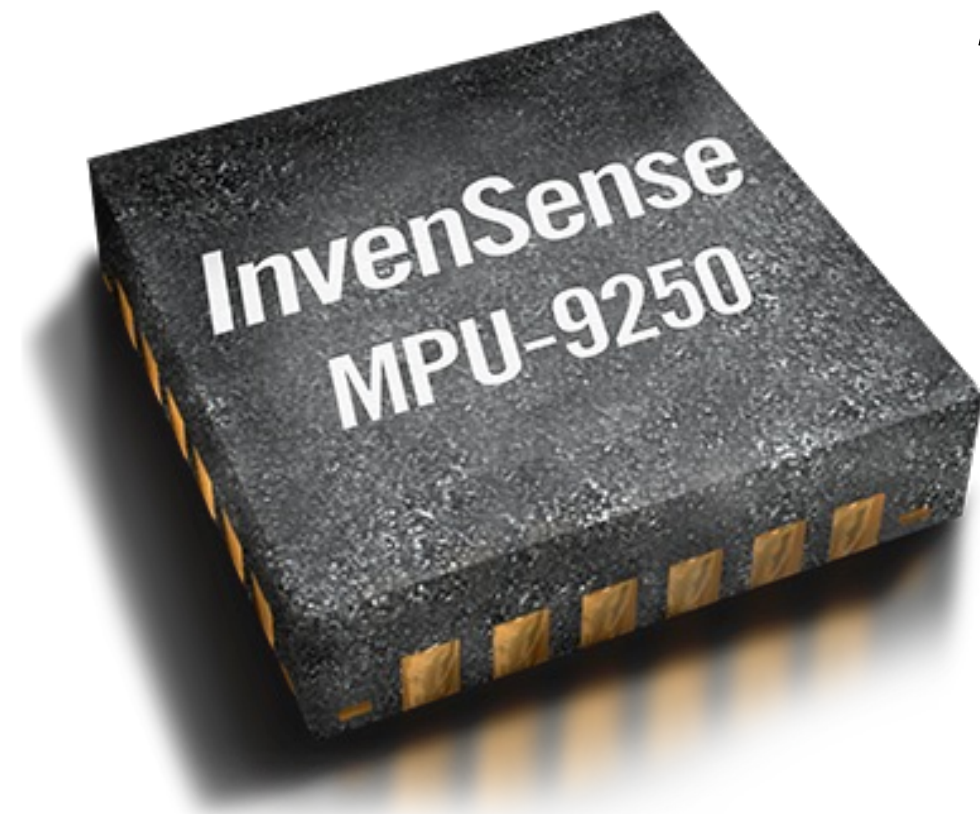
Next Wed.

IMU 2

Huaishu Peng | UMD CS | Fall 2024



Accelerometer | Gyro | Magnetometer



Invensense MPU-9250

<https://invensense.tdk.com/products/motion-tracking/9-axis/mpu-9250/>

I²C library in Arduino - Wire library

Setup

Reading a register

Updating a register

Address: 0b1101000
(0x68)



```
void loop() {  
  Wire.beginTransmission(address);  
  Wire.write(0x3B);  
  Wire.endTransmission();  
  
  Wire.requestFrom(address, 1);  
  ACCEL_XOUT_H = Wire.read();  
  
  Wire.beginTransmission(address);  
  Wire.write(0x3C);  
  Wire.endTransmission();  
  
  Wire.requestFrom(address, 1);  
  ACCEL_XOUT_L = Wire.read();  
  
  ACCEL_X_RAW = ACCEL_XOUT_H << 8 | ACCEL_XOUT_L;  
  
  gX = ACCEL_X_RAW / 16384.0;  
  
  //Plot data  
  Serial.print(">gX:");  
  Serial.println(gX);  
  delay(10);  
}
```

I²C library in Arduino - Wire library

Practice: Read temperature of the sensor in degrees C

Datasheet:

https://cdn.sparkfun.com/assets/learn_tutorials/5/5/0/MPU-9250-Register-Map.pdf

<https://invensense.tdk.com/wp-content/uploads/2015/02/PS-MPU-9250A-01-v1.1.pdf>

I²C library in Arduino

Setup

Reading a register

Updating a register

REGISTER 53 - I2C_SLV4_DI.....	27
4.18 REGISTER 54 – I ² C MASTER STATUS.....	28
4.19 REGISTER 55 – INT PIN / BYPASS ENABLE CONFIGURATION.....	29
4.20 REGISTER 56 – INTERRUPT ENABLE	29
4.21 REGISTER 58 – INTERRUPT STATUS	30
4.22 REGISTERS 59 TO 64 – ACCELEROMETER MEASUREMENTS.....	31
4.23 REGISTERS 65 AND 66 – TEMPERATURE MEASUREMENT.....	33
4.24 REGISTERS 67 TO 72 – GYROSCOPE MEASUREMENTS	33
4.25 REGISTERS 73 TO 96 – EXTERNAL SENSOR DATA.....	35
4.26 REGISTER 99 – I ² C SLAVE 0 DATA OUT.....	37
4.27 REGISTER 100 – I ² C SLAVE 1 DATA OUT.....	37
4.28 REGISTER 101 – I ² C SLAVE 2 DATA OUT.....	37
4.29 REGISTER 102 – I ² C SLAVE 3 DATA OUT.....	37
4.30 REGISTER 103 – I ² C MASTER DELAY CONTROL.....	38
4.31 REGISTER 104 – SIGNAL PATH RESET.....	39
4.32 REGISTER 105 – ACCELEROMETER INTERRUPT CONTROL	39
4.33 REGISTER 106 – USER CONTROL.....	39
4.34 REGISTER 107 – POWER MANAGEMENT 1	40
4.35 REGISTER 108 – POWER MANAGEMENT 2	41
4.36 REGISTER 114 AND 115 – FIFO COUNT REGISTERS	42
4.37 REGISTER 116 – FIFO READ WRITE	43
4.38 REGISTER 117 – WHO AM I.....	44
4.39 REGISTERS 119, 120, 122, 123, 125, 126 ACCELEROMETER OFFSET REGISTERS	44

4.23 Registers 65 and 66 – Temperature Measurement

Name: TEMP_OUT_H

Serial IF: SyncR

Reset value: 0x00 (if sensor disabled)

BIT	NAME	FUNCTION
[7:0]	D[7:0]	High byte of the temperature sensor output

Name: TEMP_OUT_L

Serial IF: SyncR

Reset value: 0x00 (if sensor disabled)

BIT	NAME	FUNCTION
[7:0]	D[7:0]	Low byte of the temperature sensor output:
		TEMP_degC = ((TEMP_OUT – RoomTemp_Offset)/Temp_Sensitivity) + 21degC Where Temp_degC is the temperature in degrees C measured by the temperature sensor. TEMP_OUT is the actual output of the temperature sensor.

3.4.2 A.C. Electrical Characteristics

Typical Operating Circuit of section 4.2, VDD = 2.5V, VDDIO = 2.5V, T_A=25°C, unless otherwise noted.

Parameter	Conditions	MIN	TYP	MAX	Units
Supply Ramp Time	Monotonic ramp. Ramp rate is 10% to 90% of the final value	0.1		100	ms
Operating Range	Ambient	-40		85	°C
Sensitivity	Untrimmed		333.87		LSB/°C

I²C library in Arduino

Setup

Reading a register

Updating a register

```
1  #include <Arduino.h>
2  #include <Wire.h>
3  const int MPU = 0x68;
4
5  const int sda = 21;
6  const int scl = 22;
7
8
9
10 void setup() {
11     Serial.begin(9600);
12     Wire.begin(sda, scl);
13 }
14
15 void loop() {
16
17     Wire.beginTransmission(MPU);
18     Wire.write(0x41);
19     Wire.endTransmission();
20
21     Wire.requestFrom(MPU, 2);
22
23     short temperature = Wire.read() << 8 | Wire.read();
24
25     Serial.println(temperature/333.87 + 21);
26
27     delay(20);
28 }
```

BIT	NAME	FUNCTION
[7:0]	D[7:0]	Low byte of the temperature sensor output: TEMP_degC = ((TEMP_OUT – RoomTemp_Offset)/Temp_Sensitivity) + 21degC Where Temp_degC is the temperature in degrees C measured by the temperature sensor. TEMP_OUT is the actual output of the temperature sensor.

3.4.2 A.C. Electrical Characteristics

Typical Operating Circuit of section 4.2, VDD = 2.5V, VDDIO = 2.5V, T_A=25°C, unless otherwise noted.

Parameter	Conditions	MIN	TYP	MAX	Units
Supply Ramp Time	Monotonic ramp. Ramp rate is 10% to 90% of the final value	0.1		100	ms
Operating Range	Ambient	-40		85	°C
Sensitivity	Untrimmed		333.87		LSB/°C

Why might we want to change the measurement range?

3.2 Accelerometer Specifications

Typical Operating Circuit of section [4.2](#), VDD = 2.5V, VDDIO = 2.5V, T_A=25°C, unless otherwise noted.

PARAMETER	CONDITIONS	MIN	TYP	MAX	UNITS
Full-Scale Range	AFS_SEL=0		±2		<i>g</i>
	AFS_SEL=1		±4		<i>g</i>
	AFS_SEL=2		±8		<i>g</i>
	AFS_SEL=3		±16		<i>g</i>
ADC Word Length	Output in two's complement format		16		bits
Sensitivity Scale Factor	AFS_SEL=0		16,384		LSB/ <i>g</i>
	AFS_SEL=1		8,192		LSB/ <i>g</i>
	AFS_SEL=2		4,096		LSB/ <i>g</i>
	AFS_SEL=3		2,048		LSB/ <i>g</i>

With a ±16*g* range:

$$\frac{1g}{2,048 \text{ steps}} = 0.00048828g \text{ per step}$$

(0.49 mg per step)

With a ±2*g* range on the same ADC:

$$\frac{1g}{16,384 \text{ steps}} = 0.00006104g \text{ per step}$$

(0.061 mg per step)

I²C library in Arduino - Wire library

Setup

Reading a register

Updating a register

Let's try to read acceleration data **in the range of $\pm 16g$**
so that we can **detect strong sudden motions!**

I²C library in Arduino - Wire library

Setup

Reading a register

Updating a register

4.7 Register 28 – Accelerometer Configuration

Serial IF: R/W
Reset value: 0x00

BIT	NAME	FUNCTION
[7]	ax_st_en	X Accel self-test
[6]	ay_st_en	Y Accel self-test
[5]	az_st_en	Z Accel self-test
[4:3]	ACCEL_FS_SEL[1:0]	Accel Full Scale Select: ±2g (00), ±4g (01), ±8g (10), ±16g (11)
[2:0]	-	Reserved

```
Wire.beginTransmission(addr) //opens communication with the slave device with its addr
Wire.write(data) //prepares to send data to addr
Wire.write(data) //prepares to send data to addr
...
Wire.endTransmission() //sends the data and returns
```

I²C library in Arduino - Wire library

Setup

Reading a register

Updating a register

```
4
5
6 void setup() {
7   Serial.begin(9600);
8   Wire.begin();
9
10  Wire.beginTransmission(MPU);
11  Wire.write(0x1C); //Talk to the ACCEL_CONFIG Register
12  Wire.write(0b00011000); //Set the register bits for 16g scale
13  Wire.endTransmission();
14 }
```

4.7 Register 28 – Accelerometer Configuration

Serial IF: R/W
Reset value: 0x00

BIT	NAME	FUNCTION
[7]	ax_st_en	X Accel self-test
[6]	ay_st_en	Y Accel self-test
[5]	az_st_en	Z Accel self-test
[4:3]	ACCEL_FS_SEL[1:0]	Accel Full Scale Select: ±2g (00), ±4g (01), ±8g (10), ±16g (11)
[2:0]	-	Reserved

Wire.beginTransmission(addr) //opens communication with the slave device with its addr
Wire.write(data) //prepares to send data to addr
Wire.write(data) //prepares to send data to addr
...
Wire.endTransmission() //sends the data and returns

I²C library in Arduino - Wire library

Setup

Sensitivity Scale Factor	AFS_SEL=0		16,384		LSB/g
	AFS_SEL=1		8,192		LSB/g
	AFS_SEL=2		4,096		LSB/g
	AFS_SEL=3		2,048		LSB/g

Reading a register

```
gX = ACCEL_X_RAW / 2048.0;
```

Updating a register

```
Wire.beginTransmission(addr) //opens communication with the slave device with its addr
Wire.write(data) //prepares to send data to addr
Wire.write(data) //prepares to send data to addr
...
Wire.endTransmission() //sends the data and returns
```

I²C library in Arduino - Wire library

Understanding Gyro Data

BIT	NAME	FUNCTION
[7:0]	D[7:0]	Low byte of the X-Axis gyroscope output GYRO_XOUT = Gyro_Sensitivity * X_angular_rate Nominal FS_SEL = 0 Conditions Gyro_Sensitivity = 131 LSB/(°/s)

degree per second

I²C library in Arduino

Understanding Gyro Data

```
gyroX_Per_S = GYRO_X_RAW/131.0;  
gyroY_Per_S = GYRO_Y_RAW/131.0;  
gyroZ_Per_S = GYRO_Z_RAW/131.0;
```

angular velocity

```
currentTime = millis();  
elapsedTime = (currentTime - previousTime) / 1000;
```

```
gyroAngleX = gyroAngleX + gyroX_Per_S * elapsedTime;  
gyroAngleY = gyroAngleY + gyroY_Per_S * elapsedTime;  
gyroAngleZ = gyroAngleZ + gyroZ_Per_S * elapsedTime;
```

I²C library in Arduino

Dead Reckoning

- Drifting over time because errors accumulated and built upon previous measurements -> data won't be accurate
- Still, we can reduce the error with a simple calibration process
- For example, you can record 10s of raw x y z gyro data to find the average offset -> which you can plug into your final output
- Offset varies based on your device

```
GYRO_X_RAW -= gyroXoffset;  
GYRO_Y_RAW -= gyroYoffset;  
GYRO_Z_RAW -= gyroZoffset;  
  
gXrate = GYRO_X_RAW / 131.0;  
gYrate = GYRO_Y_RAW / 131.0;  
gZrate = GYRO_Z_RAW / 131.0;
```